Predicting Spread, Total Points, and Total Passing Yards for Week 7-11 of 2025 NFL Games

Sarah Cao
Nick Copland
Sophia Draghici
Stephen Bridges
Daniel Larson

October 15, 2025
STOR 538
Dr. Mario Giacomazzo

1. Data Information

1.1 Data Cleaning

Our group began by exploring multiple potential sources of NFL data, including Kaggle game score datasets, Pro Football Reference statistics, and the nflfastR database. After experimenting with several of these options and loading each dataset in R for comparison, we decided that the nflfastR database was the most suitable choice. It provides a detailed play-by-play dataset that includes hundreds of variables for every game and, at the time of accessing, data up to five weeks into the 2025 season.

We accessed the data through the nflfastR R package and loaded all games from the 2024 NFL season and the first five weeks of the 2025 season. This ensured that our dataset included both a complete past season and the most recent available data for the current season, which provides a strong historical basis for model training as well as up-to-date context for prediction.

After loading both seasons, we combined them into a single dataset and filtered the data to include only regular-season games by keeping observations where the variable season_type was equal to "REG". This ensured that our dataset did not include preseason or postseason games, which can differ in strategy and player usage. The merged dataset contained 372 variables and data for over 60,000 games. These variables represent most aspects of gameplay, including game identifiers, team and player information, field position, play type, timing, scoring outcomes, and advanced analytics like expected points and win probability estimates.

To make the dataset more manageable and focused for modeling, we created a smaller version with 56 most relevant variables. This was done by printing a list of all the column names, comparing them to their field descriptions on nflfastr.com, and selecting those most useful for predicting our three outcome variables – spread, pass, and total. The variables included are game_id, old_game_id, season_type, week, game_date, home_team, away_team, posteam, posteam_type, defteam, side_of_field, yardline_100, location, stadium, qtr, game_half, quarter_end, quarter_seconds_remaining, half_seconds_remaining, game_seconds_remaining, time, yrdln, drive, down, goal_to_go, ydstogo, yards_gained, play_type, sp, shotgun, success, no_huddle, home_score, away_score, total_home_score, total_away_score, score_differential, spread_line, total_line, total, pass_attempt, rush_attempt, complete_pass, passing_yards,

rushing_yards, receiving_yards, air_yards, yards_after_catch, epa, complete_pass, epa, wp, wpa, passer_player_name, rusher_player_name, and receiver_player_name. We then used the select() function in R to subset the data with these columns and saved the cleaned dataset as a CSV file.

We created a new binary variable, home, using the mutate function where (home = ifelse(home_team == posteam, 1, 0)). Then, dataset rows were filtered into a home team dataset if the home variable was equal to 1 and an away team dataset if the home variable was equal to 0. To distinguish the variables for home and away teams, we added "home_" and "away_" respectively to the start of the column names. Using an inner_join, the home and away team datasets were merged by the game_id, week, home_team, and away_team variables.

1.2 Creation of Summary Variables

Given that the nflfastR package contains play-by-play data, we aggregated observations to create summary variables for plays from the same game. Grouping the data by game_id, season_type, week, posteam, defteam, home_team, and away_team, we created new variables by summing all the observations in the dataframe for pass_attempts, rush_attempts, passing_yards, and rushing_yards. Additionally, we created per-play statistics by computing the means of complete_pass, epa, success, pass_attempt, shotgun, yards_gained, air_yards, yards_after_catch, third downs (observations where down = 3), and ydstogo for the grouped data. Lastly, total_plays was calculated by counting the number of rows for each game.

We also created the variables Spread, Total, and Pass. Spread was computed by subtracting away points from home points. Total was computed by adding home points and away points. Pass was calculated by adding home passing yards and away passing yards.

1.3 New Variables

We decided to create new variables to improve the predictive ability of our models. Given the potential for high skewedness and variabilities for the NFL statistics we wanted to predict, new variables can help account for the interactions in the predictors and train a more accurate

model. These variables can also provide insights into the impact of player performance on team statistics.

1.3.1 Passer Rating

Particularly for predicting passing yards, the rating of the passers on each team can provide useful information. Passers influence the offensive performance of a team because of their ability to create major plays and advance the ball to score. This prompted us to look at the impact of passer ratings on team statistics.

Using the official NFL formula, we grouped by the names of passers and created five new variables by summing the observations in each variable. These include ATT = Number of passing attempts, CMP = Number of completions, YDS = Passing yards, TD = Touchdown passes, and INT = Interceptions. Each variable was scaled according to the following formulas.

$$a = \left(\frac{\text{CMP}}{\text{ATT}} - 0.3\right) \times 5$$

$$b = \left(\frac{\text{YDS}}{\text{ATT}} - 3\right) \times 0.25$$

$$c = \left(\frac{\text{TD}}{\text{ATT}}\right) \times 20$$

$$d = 2.375 - \left(\frac{\text{INT}}{\text{ATT}} \times 25\right)$$

After scaling individual components, the passer rating is calculated by the following formula.

$$\text{Passer Rating} = \left(\frac{a+b+c+d}{6}\right) \times 100$$

Additionally, if a, b, c, or d is greater than 2.375, the result is set to 2.375. If a, b, c, or d is negative, it is set to zero. This was accomplished by the mutate function in R, where we used *a=pmin(pmax(a, 0), 2.375)* for a and similarly for the remaining variables.

Based on this formula, passers with higher numbers of completions, passing yards, and touchdown passes, and a lower number of interceptions are rated more highly. The rating takes into consideration both the passer's ability to create plays and score, and any errors.

After creating this variable, the data was grouped by game_id, season_type, week, posteam, defteam, home_team, and away_team and the mean passer rating of each player was computed. This dataset was joined with the larger dataset described in Section 1.1 with a left_join by passer player name.

1.3.2 Offense Scores

Creating separate variables for offense and defense scores allows us to evaluate two important aspects of team performance in football. A balance of offensive and defensive ability is crucial for a team's success. Given the importance of the offensive line in creating plays and scoring, we decided to create a variable to score the offensive performance of different teams. The following variables were calculated by taking the average after grouping by offense team data (the variable posteam). These include success, EPA per play, yards per pass, completion number, touchdowns, yards per rush, and yards per play. For yards per pass and yards per rush, we took the mean of passing yards and rushing yards, where pass attempt was equal to one and rushing attempt was equal to 1, respectively. Then each variable was standardized by using z-scores with the scale function in R. For instance, the standardized value of EPA per play was computed by *z_epa_per_play = 1 * scale(epa_per_play)[,1]*. Finally, we summed the standardized variables to create the offensive score.

Team offensive score *= standardized success rate + standardized EPA per play + standardized completion number + standardized number of touchdowns + standardized yards per rush + standardized yards per play*

1.3.3 Defense Scores

A football team's defensive ability to prevent the opponent team from scoring, generate turnovers, and intercept further advances down the field is crucial to the team's overall outcome. Therefore, our data needed a defensive score variable to assess the defensive performance of NFL teams.

We calculated the following variables by taking the averages of several variables that correlates with defensive performance after grouping by defense team data (with the variable defteam). These include success, EPA per play, yards per pass, completion number, touchdowns, yards per rush, sacks, and yards gained. For yards per pass and yards per rush, we took the mean of passing yards and rushing yards, where pass attempt was equal to one and rushing attempt was equal to one, respectively. Then, each variable was standardized using z-scores with the scale function in R similar to the team offensive score. However, we multiplied some variables by negative 1. For instance, as the yards per play variable grouped by defense team describes the number of yards that they allowed the other team to gain, the z-score for this variable was multiplied by -1. Similarly, the z-scores for all variables except for sack rate were multiplied by -1. After, we summed the standardized variables to create the team defensive score. With this formula, lower defense scores indicate higher defensive performance.

Team defensive score = *- standardized EPA - standardized success - standardized yards per play - standardized yards per play - standardized yards per pass - standardized yards per rush - standardized completion number - standardized number of touchdowns + standardized sack number*

After creating the offensive and defensive scores, these variables were filtered into two datasets based on home teams and away teams. This dataset was joined to the main dataset (described in Section 1.1) by using a left_join by posteam and defteam, respectively.

2. Methodology for Spread Variable

2.1 Model Objective

The goal of our Spread model was to predict the outcome of upcoming NFL games as defined:

Spread = Home Points - Away Points.

To predict the spread, we began by further cleaning the data and building an aggregated table of relevant statistics for predicting the overall game performance. This involved taking the large play-by-play dataset and reducing it into game-specific rows, with each row representing a unique game. These rows included home and away statistics for indicators such as points, total yards, yards per passing play, and more, as well as the Vegas-predicted spread before the game. This would form the foundation of our modeling process, allowing us to predict spread by looking at and comparing season trends between teams while taking advantage of the Vegas predicted spread of each game.

2.2 Comparative Performance Metrics

Next, we constructed team-level season averages and differences between home and away teams. Season-to-date averages were computed for points scored, points allowed, yards gained, yards per play, and other performance metrics. Then, we calculated the differences between home and away teams for these metrics to create predictors that capture relative team strength, providing an idea of how much better one team may be than the other. The logic behind this is that spreads are better predicted by how a team performs relative to its opponent, rather than by a single team's raw statistics alone. The season averages were merged back into the game-level dataset to create a modeling dataset where each row represents a game with the team's season average differences present, along with the actual game data.

2.3 Modeling

Finally, the data was split into training and test sets, used to build a series of models and predictions were then evaluated for accuracy. The models tested included a baseline linear regression, interaction models, tree-based models, and models with weighted performances. Testing included experimentation with dropping variables that could introduce collinearity. All models were trained and validated using 2024 season data, and evaluated with mean absolute error (MAE) as the measure of predictive accuracy.

Our final model, with the lowest MAE when tested on 2024 data, was a multiple linear regression. Noticing that the average spread was being influenced by significant outliers, we examined the correlation grid of our predictor variables and implemented dampening factors to reduce the impact of extreme team performance differences. The dampening was achieved by scaling down the coefficients of highly correlated variables: points differential (0.6x), points against differential (0.5x), yards differential (0.35x), plays differential (0.4x), and yards per play differential (0.5x). The model's predictions were constrained to more realistic NFL spread ranges while maintaining predictive accuracy. This dampening approach succeeded at reducing extreme predictions (14+ points), which if incorrect would be the most punishing when being evaluated by mean absolute error, while preserving the model's ability to differentiate between close games and blowouts.

```
lm(formula = actual_spread ~ spread_line + 0.6 * points_for_diff + 0.5 *
points_against_diff + 0.35 * total_yards_diff + 0.35 * pass_yards_diff + 0.35 *
rush_yards_diff + 0.4 * plays_diff + 0.5 * yards_per_play_diff, data = train)
```

2.4 Summary

This model uses both the Vegas spread line and team statistical differences as predictors. Among the variables, points_for_diff and points_against_diff were far and away the most significant, which is unsurprising since we are predicting a points metric. Other variables including yardage and play-based differentials contributed less predictive power once scoring metrics were included, yet still reduced the MAE.

To generate predictions, this model can utilize the current season's averages and the Vegas spread line for both teams to predict a spread. This allows for an evidence-based estimate of game outcomes while incorporating both statistical performance trends and general expectations, providing a balanced spread prediction.

3. Methodology for Total Variable

3.1 Model Objective and Rationale

The goal of our Total model was to predict the total number of points scored in each NFL game, defined as:

Total = Home Points + Away Points.

Because this outcome is numeric, likely nonlinear, and influenced by interacting factors such as offensive and defensive strength, recent form, and home-field advantage, we selected a Random Forest regression approach. Random Forests are ensemble methods that average the predictions of many regression trees, capturing nonlinear and interaction effects without requiring explicit transformations.

Alternative models, such as multiple linear regression and single CART trees, were initially explored for interpretability but produced higher mean absolute error (MAE) and less stability across cross-validation folds. The Random Forest model consistently achieved the lowest out-of-sample MAE, so it was chosen as our final predictive model for Total.

3.2 Feature Engineering

Following the data-cleaning steps described in Section 1, we aggregated the play-by-play dataset to the game level using group_by() and summarise() functions. Each row in this dataset represents a completed NFL game with the variables home_points, away_points, total, home_team, away_team, season, and week.

To capture recent team performance trends, we converted the dataset into a team-game structure, listing every game twice, once from each team's perspective. Using the slider package, we computed rolling averages and lagged values for:

- pf_l3_pre / pa_l3_pre – average points for and against over the previous 3 games (lagged one game, so only prior information is used);

- pf_season_pre / pa_season_pre – cumulative season-to-date averages up to the week before the game.

From these team-level features, we created matchup-level predictors:

- Sums (pf_l3_sum, pa_l3_sum, pf_season_sum, pa_season_sum) measuring the combined offensive and defensive strength of both teams;
- Differences (pf_l3_diff, pa_l3_diff) capture the relative form between the teams.

These engineered variables provide a dynamic snapshot of offensive and defensive momentum entering each game and help model nonlinear effects such as "hot" offenses or fatigued defenses.

3.3 Model Specification and Tuning

We implemented the model in R (version 4.4) using the tidymodels and ranger packages. The prediction formula was:

Total ~ *season + week + home_team + away_team + home_pf_l3 + away_pf_l3 + home_pa_l3 + away_pa_l3 + home_pf_season + away_pf_season + home_pa_season + away_pa_season + pf_l3_sum + pa_l3_sum + pf_season_sum + pa_season_sum + pf_l3_diff + pa_l3_diff.*

All categorical variables (teams) were dummy-encoded, and numeric variables were median-imputed if missing.

Model tuning was performed through 3-fold cross-validation, minimizing MAE.

The hyperparameters searched included:

- mtry – number of predictors randomly sampled at each split, centered around $\sqrt{p}$;
- min_n – minimum node size, tested at 3 and 7;
- trees = 400;

- importance = "permutation" (for later interpretation).

The best configuration was selected automatically using select_best(metric = "mae").

3.4 Model Training and Validation

After preprocessing and tuning, the finalized workflow was trained on all available regular-season data through Week 6 of 2025. Cross-validation confirmed low error variance across folds, and permutation importance indicated that recent offensive averages (pf_l3_pre, pf_season_pre) and recent defensive performance (pa_l3_pre) were the most influential predictors, while week number and team identifiers contributed less.

3.5 Prediction for Future Games

To produce out-of-sample forecasts, we created a separate Predictions.csv file containing all matchups from Weeks 7–11 (October 15 – November 18, 2025) using the nflreadr::load_schedules() function.

For these games, we computed pre-game lagged features using the same logic as the training data, ensuring that no information was leaked from future outcomes.

The final model was then applied via:

predict(final_fit, new_data = pred_df)

Predicted values were inserted into the Total column of Predictions.csv. This automated pipeline ensures reproducibility and guarantees that all predictions are numeric and based solely on information known before game day.

3.6 Summary

In summary, our Random Forest model for Total points leverages rolling averages of team offensive and defensive performance to model nonlinear scoring dynamics. Through

cross-validation and hyperparameter tuning, the approach minimized prediction error while remaining interpretable and reproducible. The final output, a completed Predictions.csv file containing numeric Total predictions for Weeks 7–11, serves as the foundation for evaluating our model's predictive accuracy under the MAE scoring framework described in the project guidelines.

4. Methodology for Pass Variable

4.1 Model Objective and Rationale

The goal of our Pass model was to predict the total number of passing yards in each NFL game, defined as:

Pass = Home Passing Yards + Away Passing Yards.

Because this variable is likely to be skewed, we considered a variety of different models and utilized new variables created, including defensive score, offensive score, and passer rating score. To evaluate the predictive ability of our model, we decided to use a method involving cross-validation to test the model on unseen data. We also split the data into training and testing datasets to assess the accuracy and efficiency of our model and test its generalizability. These steps aimed to reduce overfitting and produce a model with high prediction accuracy.

4.2 Linear Regression Model and stepAIC for Best Model

To avoid overfitting and balance fit with model complexity, we decided to create a linear regression model and run a stepwise regression on the model in both directions. To accomplish this, we ran a linear regression model in R with pass as the response variable and all other variables in the complete dataset without game_id as predictors. The categorical variables were recoded into dummy variables, and the numeric variables were median-imputed if missing. Then, with the library MASS, we used the stepAIC function to identify the best model.

```
lm_model <- lm(pass ~ . -season, data = nflstat2)

lm_model <- stepAIC(lm_model, direction = "both", trace=0)
```

```
Call:
lm(formula = pass ~ week + home_pass_attempts + home_rush_attempts +
    home_completion_rate + home_success_rate + home_total_plays +
    home_pass_play_pct + home_shotgun_pct + home_avg_yards_per_play +
    home_avg_air_yards + home_avg_yac + home_total_rushing_yards +
    home_third_down_rate + home_avg_ydstogo + away_pass_attempts +
    away_epa_per_play + away_success_rate + away_total_plays +
    away_pass_play_pct + away_avg_yards_per_play + away_avg_air_yards +
    away_avg_yac + away_total_rushing_yards + away_avg_ydstogo +
    home_defensive_score, data = nflstat2)

Residuals:
   Min     1Q Median     3Q    Max
-62.01 -12.57  -1.09  10.57 105.66

Coefficients:
                           Estimate Std. Error t value Pr(>|t|)
(Intercept)              -800.63763   34.40313 -23.272  < 2e-16
week                        0.13350    0.08631   1.547 0.122144
home_pass_attempts         -1.34371    0.52857  -2.542 0.011119
home_rush_attempts         -0.62762    0.21949  -2.859 0.004303
home_completion_rate       46.81500   22.61849   2.070 0.038648
home_success_rate          17.64259   10.53044   1.675 0.094070
home_total_plays            4.79741    0.26214  18.301  < 2e-16
home_pass_play_pct        131.52251   45.22974   2.908 0.003693
home_shotgun_pct          -10.05876    4.13253  -2.434 0.015049
home_avg_yards_per_play    69.99103    1.22255  57.250  < 2e-16
home_avg_air_yards          1.54983    0.33677   4.602 4.54e-06
home_avg_yac                1.64458    0.38807   4.238 2.40e-05
home_total_rushing_yards   -0.89194    0.02093 -42.614  < 2e-16
home_third_down_rate      -28.28855   20.03474  -1.412 0.158168
home_avg_ydstogo            3.00412    0.85876   3.498 0.000482
away_pass_attempts         -1.14779    0.54014  -2.125 0.033752
away_epa_per_play         -10.30479    5.61993  -1.834 0.066913
away_success_rate          38.42397   11.66701   3.293 0.001013
away_total_plays            4.31125    0.25366  16.996  < 2e-16
away_pass_play_pct        161.13918   43.73931   3.684 0.000238
away_avg_yards_per_play    70.71842    0.95893  73.747  < 2e-16
away_avg_air_yards          1.11416    0.25117   4.436 9.85e-06
away_avg_yac                1.32076    0.33938   3.892 0.000104
away_total_rushing_yards   -0.90825    0.01590 -57.113  < 2e-16
away_avg_ydstogo            4.86040    0.78784   6.169 8.85e-10
home_defensive_score       -0.20189    0.09618  -2.099 0.035986
```

Summary of the Best Model from stepAIC

The predictor variables identified and the pass and date variables were put in a new dataset.

4.3 Training and Testing Sets

To create the training dataset, we added more data from the 2020-2023 seasons to better train our model. For the testing dataset, we filtered the data to observations from the first few weeks of the 2025 season. We believed the split was appropriate given our goal of predicting total passing yards in weeks 7-11 of the NFL season.

4.4 Model Creation

For the Pass variable, we decided the best model to use was a gradient boosting model as it can capture non-linear relationships and uses multiple decision trees. It also utilizes shrinkage to reduce overfitting of the model. As such, the gradient boosting model would provide an efficient and accurate prediction of the pass variable.

To train the gradient boosting model, we used the training data with pass as the response variable and all other variables identified when running the stepAIC as predictors. The code below describes the gradient boosting model used.

```
gbm_model <- gbm(
  formula = pass ~ .,
  data = train_data,
  distribution = "gaussian",
  n.trees = 5000,
  interaction.depth = 3,
  shrinkage = 0.01,
  cv.folds =5)
```

The number of trees to use when predicting on the test data was determined by the code: best <- gbm.perf(gbm_model, method = "cv"), which gave us an output of 4991 trees.

We utilized this model to generate predictions on the test data using the following code:

```
predictions <- predict(gbm_model, newdata = test_data, n.trees = gbm_model$best)
```

The final model selected had the lowest RMSE and MAE of 22.49971 and 18.036, respectively.

4.5 Prediction on Future Games

To generate predictions on future games, we first aggregated the averages for each team from the predictor variables and created a prediction dataframe. In this prediction dataframe, we filled the first column with NA values and updated it with the averages. Using the trained gradient boosting model, prediction dataframe, and the optimal number of trees, we predicted out-of-sample data for NFL games with the following code.

```
best <- gbm.perf(gbm_model, method = "cv", plot.it = FALSE)

prediction <- predict(gbm_model, newdata = pred_df, n.trees = best)
```

4.6 Summary

In summary, our gradient boosting model utilizes significant predictors selected by a stepwise regression to identify the linear regression model with the lowest AIC value. Through cross-validation and hyperparameter tuning, the model minimized MAE while maintaining predictive accuracy for the Pass variable.